

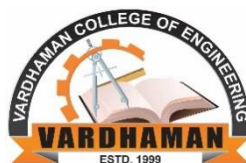
COURSE END PROJECT
E-COMMERCE INVENTORY MANAGEMENT SYSTEM
ADVANCED DATA STRUCTURES LABORATORY (A8513)
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

24885A0549	N.SRIJITH
24885A0551	C.SUJAL
24885A0553	D.STEVEN SON

UNDER THE GUIDANCE OF

Mr. Naresh Goud M
Assistant Professor, Dept of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VARDHAMAN COLLEGE OF ENGINEERING
(AUTONOMOUS)

Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,
UGC "College with Potential for Excellence" and ISO 9001:2015 Certified
Shamshabad – 501 218, Hyderabad

NOVEMBER, 2024



VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,
UGC "College with Potential for Excellence" and ISO 9001:2015 Certified

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course End Project

ADVANCED DATA STRUCTURES LABORATORY (A8513)

Check List

S. No.	Content	Status (✓ / ×)
1	Abstract	✓
2	Objectives	✓
3	Problem Statement	✓
4	Algorithm	✓
5	Flow Chart/ Data Set	✓
6	Source Code	✓
7	Input	✓
8	Output	✓
9	Conclusion	✓
10	Future Scope	✓
11	References	✓

Signature of the Guide

Mr. Naresh Goud M
Assistant Professor
Dept. of CSE

Course End Project

ADVANCED DATA STRUCTURES LABORATORY (A8513)

Table of Content

S. No	Content	Page. No
1	Abstract	1
2	Objectives	2
3	Problem Statement	3
4	Algorithm	4
5	Flow Chart/ Data Set	8
6	Source Code	9
7	Input	15
8	Output	16
9	Conclusion	17
10	Future Scope	18
11	References	19

1. ABSTRACT

This project demonstrates practical applications of advanced data structures in real-world scenarios and emphasizes efficient memory utilization, search optimization, and modular design. The solution is adaptable for small-scale e-commerce startups or as a foundation for advanced inventory management systems.

Efficient inventory management is a cornerstone of successful e-commerce platforms, ensuring optimal stock levels and quick access to product information. This project presents an Inventory Management System implemented in C, leveraging Binary Search Trees (BSTs) and hashing for efficient organization and search functionality.

The system integrates a hash table to store products in buckets, with each bucket containing a BST for managing collisions. This combination provides efficient insertion and search operations, achieving an average-case complexity of $O(1)$ for lookups and a structured view of inventory through the BST's in-order traversal.

2. OBJECTIVES

- **Efficient Product Storage and Retrieval:**

Implement a system that can store and retrieve product information quickly and efficiently, utilizing hashing for fast lookups and Binary Search Trees (BSTs) for sorted organization and collision resolution.

- **Dynamic Inventory Management & Organized and Sorted Data Structure:**

Provide functionalities to add new products, update stock levels, and remove products from the inventory. Use a Binary Search Tree to keep product data in a sorted order.

- **Collision Handling in Hashing:**

Implement a collision handling mechanism (through chaining with BSTs) in the hash table to ensure data consistency and prevent performance degradation as the number of products increases.

- **User-Friendly Interface:**

Design a simple, menu-driven interface that allows users to easily interact with the system. The system should support key operations like adding, searching, updating, and displaying inventory.

- **Scalable and Efficient Solution:**

Ensure the system can handle a large number of products and is scalable, providing $O(1)$ average-case time complexity for lookups and efficient performance even as the inventory grows.

- **Real-World Application:**

Build a system that can be practically used by small businesses or adapted for larger e-commerce platforms, showcasing the application of data structures like hashing and BSTs in solving real-world inventory management challenges.

3. PROBLEM STATEMENT

Efficient inventory management is a critical requirement for e-commerce platforms and businesses to ensure accurate stock tracking, quick product retrieval, and seamless updates. Traditional methods or poorly optimized systems can lead to challenges such as delayed lookups, data inconsistencies, and inefficiencies in managing large datasets

4. ALGORITHM

Hashing is used for fast product lookups by storing products in a hash table. The core algorithm here is the hash function and the collision resolution technique.

The Binary Search Tree (BST) is used to maintain products in a sorted order within each hash table bucket. This allows efficient searching, insertion, and deletion operations.

Example :

1. Add Product to Inventory

Input: Product details (ID, Name, Quantity, Price, etc.)

Output: Product is stored in the hash table and/or BST.

Algorithm:

- Compute the hash value of the product's ID using a hash function:

$\text{HashIndex} = \text{HashFunction}(\text{ProductID})$

If the slot in the hash table is empty, insert the product at that index.

- If a collision occurs (the slot is already occupied):

Use a collision resolution strategy (e.g., chaining or open addressing).

- If chaining is used:

Insert the product into the linked list or BST at that hash index.

Display a success message upon successful insertion.

2. Search for a Product

Input: Product ID.

Output: Product details if found; otherwise, display "Product not found."

Algorithm:

Compute the hash value using the product ID.

- Check the hash table at the computed hash index:

If the slot is empty, return "Product not found."

- If the slot contains one or more products:

Traverse the linked list or BST at that index.

Compare each product's ID with the input ID.

If a match is found, return the product details.

If no match is found, return "Product not found."

3. Update Product Quantity

Input: Product ID, Quantity to add or subtract.

Output: Updated inventory with new quantity.

Algorithm:

Search for the product using the Search Product algorithm.

- If the product is found:

Update the quantity based on the input (add or subtract).

Ensure the new quantity is not negative.

If the product is not found, display an error message.

Display the updated product details.

4. Delete Product from Inventory

Input: Product ID.

Output: Product is removed from the inventory.

Algorithm:

Compute the hash value using the product ID.

- Check the hash table at the computed hash index:

If the slot is empty, display "Product not found."

- If the slot contains one or more products:

Traverse the linked list or BST at that index.

Search for the product with the given ID.

- If the product is found:

Remove it from the linked list or BST.

Display a success message upon deletion.

5. Display All Products

Input: None.

Output: List of all products in the inventory.

Algorithm:

- Traverse each index of the hash table:
- If the index is not empty:

Traverse the linked list or BST at that index.

Print the details of each product.

If the hash table is empty, display "No products in inventory."

6. Collision Resolution Using Chaining

Input: Hash index, product details.

Output: Product is inserted into the linked list or BST at the hash index.

Algorithm:

- Check if the hash index already contains a linked list or BST:

If no list/tree exists, create a new linked list or BST node.

- If a list/tree exists:

Insert the new product into the structure using BST insertion logic (for trees).

Ensure the structure remains organized for efficient searching.

7. Traversal of Binary Search Tree (In-Order)

Input: Root of the BST.

Output: Products displayed in sorted order by ID.

Algorithm:

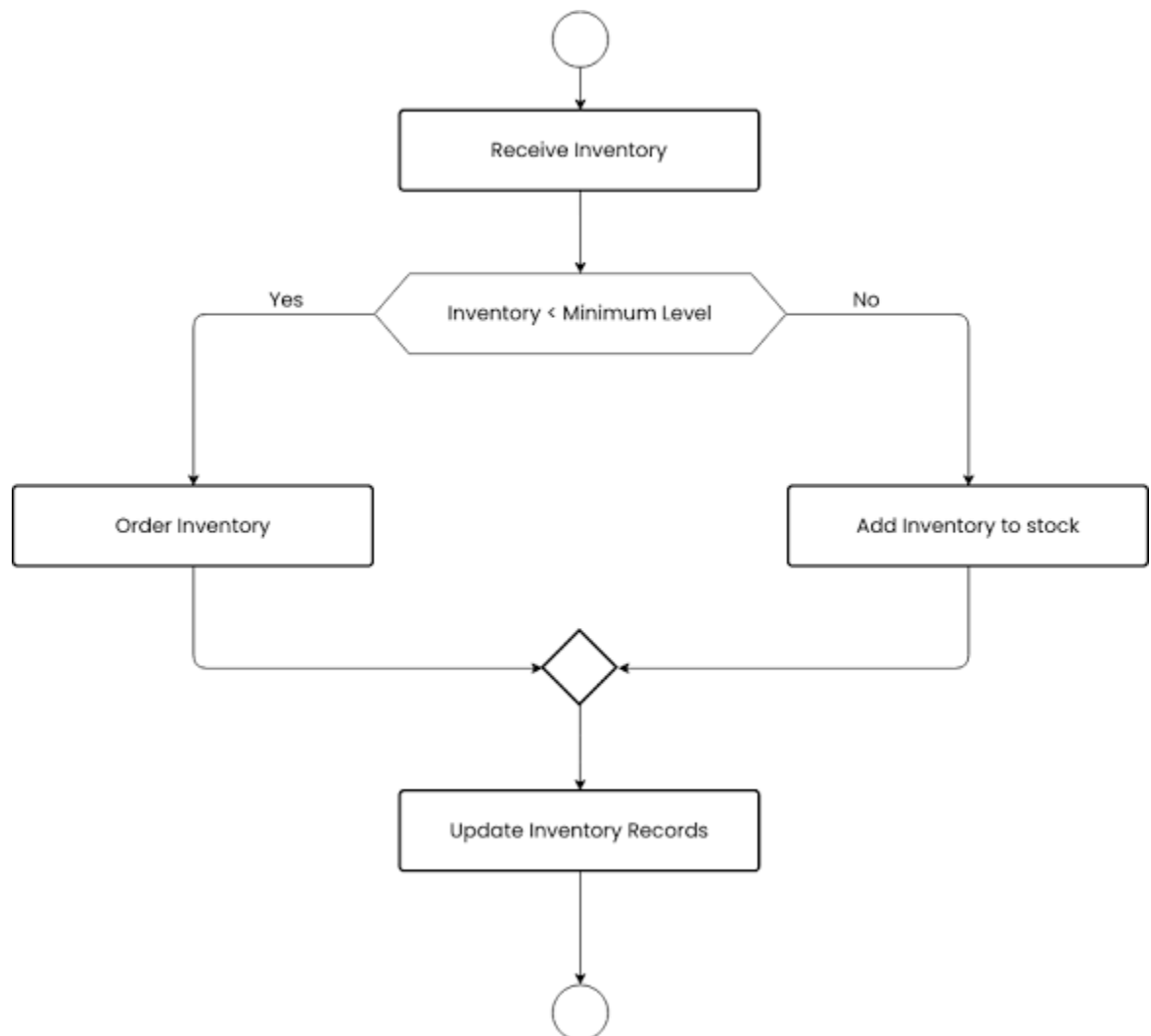
- If the tree is not empty:

Recursively traverse the left subtree.

Visit the root node (display product details).

Recursively traverse the right subtree.

5. FLOW CHART



6. SOURCE CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define HASH_TABLE_SIZE 10

// Define a structure for a BST node
typedef struct Node {
    char productName[50];
    int productID;
    int stockQuantity;
    struct Node* left;
    struct Node* right;
} Node;

// Define a structure for the hash table
typedef struct HashTable {
    Node* table[HASH_TABLE_SIZE];
} HashTable;

// Function to create a new BST node
Node* createNode(char* productName, int productID, int stockQuantity) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->productName, productName);
    newNode->productID = productID;
    newNode->stockQuantity = stockQuantity;
    newNode->left = newNode->right = NULL;
```

```

    return newNode;
}

// Insert a node into the BST

Node* insertBST(Node* root, char* productName, int productID, int stockQuantity) {
    if (root == NULL) {
        return createNode(productName, productID, stockQuantity);
    }
    if (productID < root->productID) {
        root->left = insertBST(root->left, productName, productID, stockQuantity);
    } else if (productID > root->productID) {
        root->right = insertBST(root->right, productName, productID, stockQuantity);
    }
    return root;
}

// Search for a node in the BST

Node* searchBST(Node* root, int productID) {
    if (root == NULL || root->productID == productID) {
        return root;
    }
    if (productID < root->productID) {
        return searchBST(root->left, productID);
    }
    return searchBST(root->right, productID);
}

```

```

// In-order traversal of BST (to display inventory in sorted order)

void inorderTraversal(Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);

        printf("Product ID: %d, Name: %s, Stock: %d\n", root->productID, root->productName, root->stockQuantity);

        inorderTraversal(root->right);
    }
}

// Hash function

int hashFunction(int productID) {
    return productID % HASH_TABLE_SIZE;
}

// Initialize the hash table

void initializeHashTable(HashTable* ht) {
    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        ht->table[i] = NULL;
    }
}

// Insert a product into the hash table

void insertHashTable(HashTable* ht, char* productName, int productID, int stockQuantity) {
    int index = hashFunction(productID);

    ht->table[index] = insertBST(ht->table[index], productName, productID, stockQuantity);
}

```

```

// Search for a product in the hash table

Node* searchHashTable(HashTable* ht, int productID) {

    int index = hashFunction(productID);

    return searchBST(ht->table[index], productID);

}


// Display all inventory

void displayInventory(HashTable* ht) {

    for (int i = 0; i < HASH_TABLE_SIZE; i++) {

        printf("Hash Index %d:\n", i);

        inorderTraversal(ht->table[i]);

        printf("\n");

    }

}


// Menu function

void menu() {

    printf("\n--- Inventory Management System ---\n");

    printf("1. Add Product\n");

    printf("2. Search Product\n");

    printf("3. Display Inventory\n");

    printf("4. Exit\n");

    printf("Choose an option: ");

}

```

```

int main() {
    HashTable ht;
    initializeHashTable(&ht);

    int choice, productID, stockQuantity;
    char productName[50];
    Node* result;

    while (1) {
        menu();
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter Product Name: ");
                scanf("%s", productName);
                printf("Enter Product ID: ");
                scanf("%d", &productID);
                printf("Enter Stock Quantity: ");
                scanf("%d", &stockQuantity);
                insertHashTable(&ht, productName, productID, stockQuantity);
                printf("Product added successfully.\n");
                break;

            case 2:
                printf("Enter Product ID to Search: ");

```

```

scanf("%d", &productID);

result = searchHashTable(&ht, productID);

if (result) {

    printf("Product Found: ID: %d, Name: %s, Stock: %d\n", result->productID, result-
>productName, result->stockQuantity);

}

else{

    printf("Product not found.\n");

}

break;

case 3:

    printf("--- Current Inventory ---\n");

    displayInventory(&ht);

    break;

case 4:

    printf("Exiting system. Goodbye!\n");

    exit(0);

default:

    printf("Invalid option. Please try again.\n");

}

}

return 0;

}

```

7.INPUT

--- Inventory Management System ---

1. Add Product
2. Search Product
3. Display Inventory
4. Exit

Choose an option: 1

Enter Product Name: poco

Enter Product ID: 33

Enter Stock Quantity: 5

Product added successfully.

--- Inventory Management System ---

1. Add Product
2. Search Product
3. Display Inventory
4. Exit

Choose an option: 1

Enter Product Name: vivo

Enter Product ID: 77

Enter Stock Quantity: 55

Product added successfully.

8. OUTPUT

--- Current Inventory ---

Hash Index 0:

Hash Index 1:

Product ID: 11, Name: realme, Stock: 22

Hash Index 2:

Product ID: 22, Name: redmi, Stock: 22

Hash Index 3:

Product ID: 33, Name: poco, Stock: 5

Hash Index 4:

Hash Index 5:

Hash Index 6:

Hash Index 7:

Product ID: 77, Name: vivo, Stock: 55

Hash Index 8:

Hash Index 9:

9. CONCLUSION

The Inventory Management System developed using Binary Search Trees (BST) and Hashing in C provides an efficient and scalable solution to track and manage product inventories in an e-commerce or retail environment. By utilizing advanced data structures, this system ensures that product data can be stored, searched, and updated quickly and accurately, ensuring that inventory is always displayed in an organized manner based on product IDs, allowing for efficient searches and updates.

10. FUTURE SCOPE

- **Automated Stock Replenishment:** Implement logic to track low-stock products and automatically generate purchase orders to suppliers when inventory reaches a threshold.
- **Barcode/QR Code Scanning:** Integrate barcode or QR code scanning for quicker product entry and tracking. This would be useful for managing large inventories in warehouses or retail stores.
- **Multi-Warehouse Support:** Allow inventory tracking across multiple locations or warehouses, with functionalities like transferring stock between warehouses.
- **Sales and Profit Analysis:** Incorporate sales data and profitability calculations to help businesses monitor the financial health of their inventory.
- **Real-time Notifications:** Notify users of low stock levels, upcoming expiration dates, or sales trends via email or app notifications.

11.REFERENCES

1. GeeksforGeeks - Hashing in Data Structures

This tutorial provides a detailed explanation of hashing, including the concepts of hash tables, hash functions, and collision resolution strategies.

Link:<https://www.geeksforgeeks.org/open-addressing-collision-handling-technique-in-hashing/>

2. GeeksforGeeks - Binary Search Tree (BST)

A step-by-step tutorial that covers the creation, insertion, deletion, and traversal methods for binary search trees. The tutorial includes C code examples to help you implement BSTs in your project.

Link:<https://www.geeksforgeeks.org/binary-search-tree-data-structure/>